8051 Microcontroller

Chapter 3: Loops and Jumps

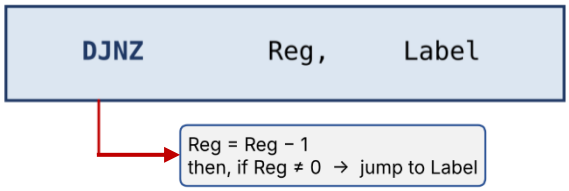
1. Loops

A loop repeats a set of instructions a certain number of times.

In the 8051, a loop is normally built from two ingredients:

- A **register** used as a counter.
- **DJNZ** — "Decrement and Jump if not Zero instruction"

The DJNZ Instruction



DJNZ decrements the named register by 1, then — if the result is not zero — jumps back to the given label. If the result is zero, the execution exits the loop.

Example: Write a code to add 5 to A, 8 Times

```
        ORG   0000H
        MOV   A, #0         ; A = 0   (same as CLR A)
        MOV   R2, #8        ; R2 = 8  (loop counter)
NEXT:   ADD   A, #5         ; repeated 8 times
        DJNZ  R2, NEXT      ; decrement R2; loop while R2 ≠ 0
        SJMP  $             ; stay here
        END
```

What are the contents of R2 and A at the end?
R2 = 0 (the counter always ends at zero — that is what stops the loop). A = 0 + 5 + 5 + 5 + 5 + 5 + 5 + 5 + 5 = 5 × 8 = 40 decimal = 28H.

©2026 Dr. Fadi Alzhouri

Recall CLR Instruction

CLR clears (sets to 0) an accumulator or any bit, such as the carry flag:

Form	Effect
`CLR A`	clears the accumulator (A = 0)
`CLR bit`	clears the bit (bit = 0)

Example:

```
        MOV   A, #55H        ; A = 0101 0101
        CLR   A              ; A = 0000 0000 = 00H
        CLR C                ; CY = 0 clear the carry flag
```

The CPL Instruction (Complement)

CPL inverts (complements) every bit of its operand — each 0 becomes a 1, and each 1 becomes a 0. This is the 1's complement operation. Like CLR, CPL comes in two forms:

Form	Effect
`CPL A`	complements every bit of the accumulator
`CPL bit`	complements a single addressable bit (e.g., CPL P1.0 toggles that I/O pin)

Example — Complementing the Accumulator

```
        MOV   A, #55H        ; A = 0101 0101
        CPL   A               ; A = 1010 1010 = AAH
```

Bit by bit: 0101 0101 (55H) → 1010 1010 (AAH). Every 0 flipped to 1 and every 1 flipped to 0.

Key Properties

- Applying CPL twice in a row restores the original value: CPL A; CPL A leaves A unchanged.
- More generally, an even number of CPL operations on the same operand returns it to its original value; an odd number leaves it complemented.
- CPL bit is commonly used to toggle a single I/O pin on and off — for example, CPL P1.0 flips an LED connected to that pin between on and off each time it executes.

©2026 Dr. Fadi Alzhouri

Maximum Loop Count with an 8-bit Register

Question: what is the maximum number of times a loop can repeat using a single 8-bit register as a counter?

```
        MOV   R2, #0
LOOP:   ADD   A, #5          ; runs …… times
        DJNZ  R2, LOOP
```

The answer is ……

Why 256, not 255? DJNZ decrements before testing. Starting R2 at 0, the first DJNZ makes R2 = 0 − 1 = 255 (FFH) — which is not zero, so it loops. Counting down from 255 to 0 gives 256 passes through the loop body in total.

Example: Write a code to complement A, 1000 Times.

Task: load A with 55H, then complement A exactly 1000 times. Since a single 8-bit register can count at most 256 times, 1000 repetitions require two nested loops whose counts multiply to 1000:

Inner × Outer	Valid?
50 × 20 = 1000	
100 × 10	
25 × 40	
250 × 4	
500 × 2	

```
        ORG   0000H
        MOV   A, #55H        ; A = 55H
        MOV   R3, #4         ; outer-loop counter
NEXT:   MOV   R4, #250       ; inner-loop counter (reloaded each pass)
LOOP:   CPL   A              ; complement A      ┤
        DJNZ  R4, LOOP       ; inner loop: 250 ×  ┘  250 × 4 = 1000
        DJNZ  R3, NEXT       ; outer loop: 4 ×
        SJMP  $
        END
```

Tracing the Result

A = 55H = 0101 0101. Complementing it once gives 1010 1010 = AAH.

Value	Binary	Hex	Complements so far
Original A	0101 0101	55H	0 (even)
After 1 x CPL	1010 1010	AAH	1 (odd)
After 2 x CPL	0101 0101	55H	2 (even)
After 3 x CPL	1010 1010	AAH	3 (odd)
…	…	…	… … …
After 1000 x CPL	0101 0101	55H	1000 (even)

Key idea: Each CPL A flips every bit. An odd number of complements leaves A flipped (AAH); an even number returns A to its original value. Since 1000 is even, A ends back at its starting value: **A = 55H.**

2. Jump

The JUMP instructions fall into two families: unconditional jumps, which always branch, and conditional jumps, which branch only if a condition is met (covered in Section 3). All conditional jumps use a relative address; unconditional jumps come in three forms with different addressing reach.

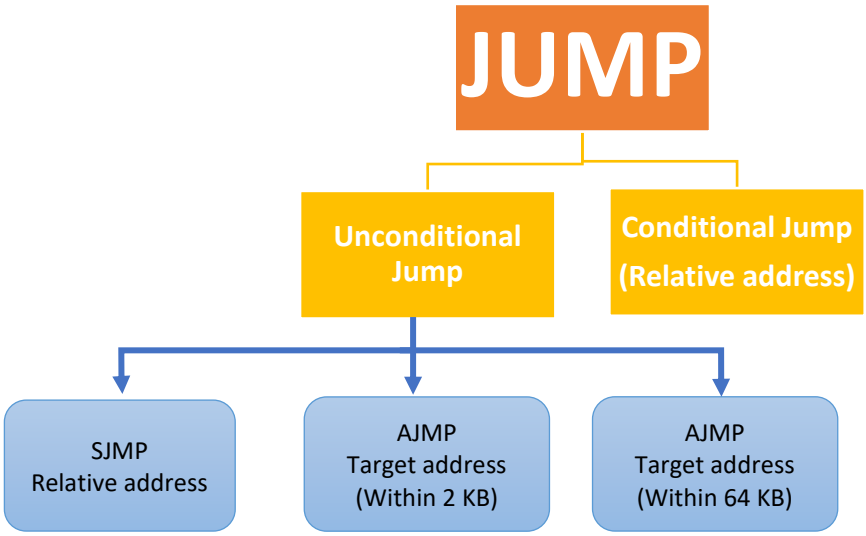


Figure 1. The JUMP instruction family.

SJMP, AJMP, and LJMP Address Ranges

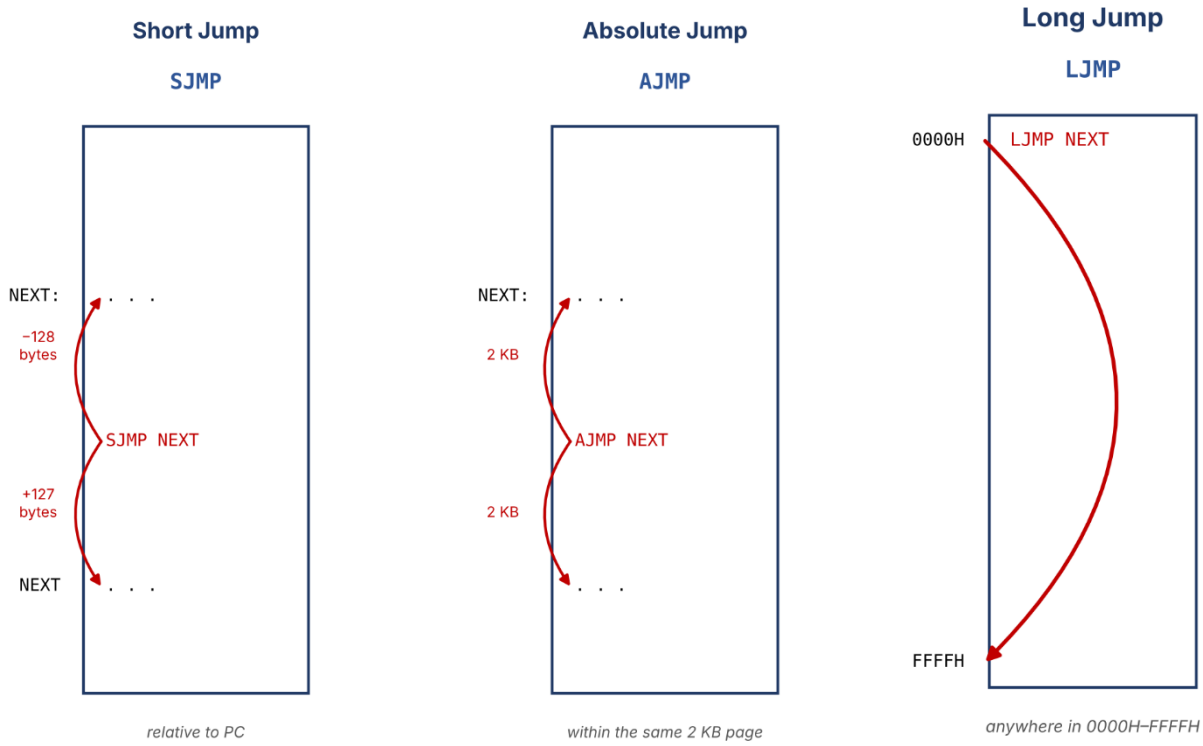


Figure 2. Reach of each unconditional jump instruction relative to its own position in memory.

1. SJMP — Short Jump

- 2-byte instruction.
- First byte: opcode (80H).
- Second byte: the relative address — 1 byte (8-bit).

The relative address range 00H–FFH is split into a forward +127 bytes and a backward −128 bytes, relative to the program counter (PC).

2. LJMP — Long Jump

- 3-byte instruction.
- First byte: opcode (02H).
- Second and third bytes: the target address — 2 bytes (16-bit).

Target-address range: 0000H–FFFFH. This lets LJMP reach any location in the full $2^{16} = 2^6 \times 2^{10}$ = 64 KB address space.

3. AJMP — Absolute Jump

AJMP encodes an 11-bit address, so it can jump anywhere within the same 2 KB page as the instruction following AJMP — shown above as roughly ±2 KB.

Is SJMP faster than LJMP? No — both execute in 2 machine cycles, even though LJMP is a larger (3-byte) instruction. SJMP's advantage is code size, not speed.

2.1 Conditional Jump

Important: every conditional jump instruction uses a relative address (like SJMP) — none of them can reach the full 64 KB address space directly.

#	Instruction	Jumps to the label if…
1	`JZ  label`	`A = 0`
2	`JNZ  label`	`A ≠ 0`
3	`JC  label`	`CY = 1`
4	`JNC  label`	`CY = 0`
5	`JB  bit, label`	`Jump if the named Bit = 1`
6	`JNB  bit, label`	`Jump if the named Bit = 0`
7	`JBC  bit, label`	`the named Bit = 1 – and then clears that bit`
8	`DJNZ  reg, label`	`Jump if the reg (after being decremented) ≠ 0  (Section 1)`
9	`CJNE  ...`	`Compare and Jump if Not Equal`

Examples

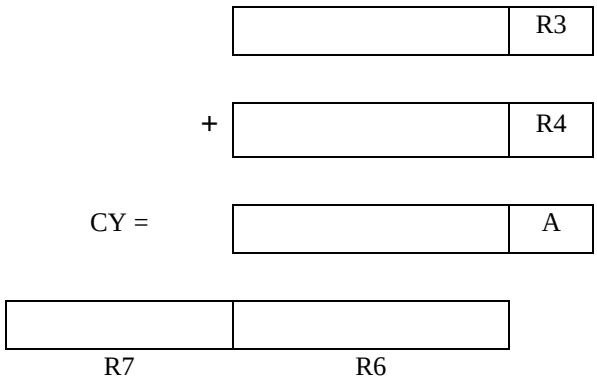
Example 1: Testing a Register for Zero

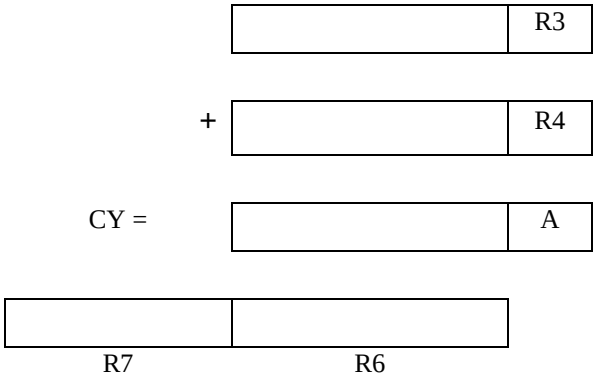
Task: Write a program to check the contents of R7. If R7 = 0, save 'Y' in R1; otherwise (R7 ≠ 0), save 'N' in R1.

```
        MOV   A, R7
        JZ    NEXT          ; if R7 = 0, skip ahead
        MOV   R1, #'N'      ; R7 ≠ 0  →  R1 = 'N'
        SJMP  HERE
NEXT:   MOV   R1, #'Y'      ; R7 = 0  →  R1 = 'Y'
HERE:   SJMP  HERE
        END
```

Example 2: Adding Two Registers with Carry

Task: Write a program to add the contents of R3 and R4; save the result in R6 (low byte) and R7 (high byte), since the 8-bit sum may overflow into a 9th bit (the carry flag).





Trace 2: R3 = 02H, R4 = 03H

R3	R4	CY	A	R7	R6
02H	03H	0	05H	00H	05H

Trace 1: R3 = FFH, R4 = FFH

R3	R4	CY	A	R7	R6
FFH	FFH	1	FEH	01H	FEH

General program (works for any R3, R4):

```
        ORG     0000H
        MOV     R7, #00         ; high byte starts at 0
        MOV     A, R3
        ADD     A, R4
        JNC     NEXT            ; if CY = 0, no overflow – skip ahead
        INC     R7              ; CY = 1  →  high byte = 1
NEXT:   MOV     R6, A           ; low byte = A
        SJMP    $
        END
```